

РТУ МИРЭА
ИПТИП
Программирование корпоративных систем

ПРАКТИЧЕСКОЕ ЗАДАНИЕ №4
СИСТЕМЫ ТЕСТИРОВАНИЯ

NetworkPacketSniffer

Тестирование сетевого анализатора трафика

Дисциплина:	Программирование корпоративных систем
Студент:	Фомин Владимир Родионович
Группа:	ЭФБО-06-24
Преподаватель:	Сокунов Дмитрий Антонович
Семестр:	Весенний семестр 2025/2026 учебного года

Москва — 2026

1. ВВЕДЕНИЕ

В рамках практического задания №4 к проекту **NetworkPacketSniffer** добавлена полноценная система автоматического тестирования. Тесты написаны с использованием фреймворка **GoogleTest v1.14.0** и охватывают все ключевые компоненты: разбор пакетов, захват, статистику, генерацию отчёта и консольный интерфейс.

Все **57 тестов** (53 модульных + 4 сценарных) проходят за 5.7 секунды и запускаются единой командой `ctest --test-dir build` в рамках той же системы сборки CMake.

Задачи практического задания

- подключить GoogleTest через механизм CMake FetchContent;
- добавить директорию `tests/` с тестовыми файлами (6 unit + 4 сценария);
- написать 5–7 тестов для каждого класса — корректный ввод, некорректный ввод, граничные случаи, пустые данные, неподдерживаемые протоколы;
- обеспечить запуск командой `ctest --test-dir build`.

2. ФРЕЙМВОРК GOOGLETEST

GoogleTest — открытый C++-фреймворк для модульного тестирования (Google, 2008). Предоставляет макросы `TEST` / `TEST_F`, богатый набор `EXPECT_*` / `ASSERT_*`-утверждений и интеграцию с CTest через `gtest_discover_tests()`.

Подключение через CMake FetchContent

Библиотека скачивается и собирается автоматически при первой конфигурации — установка в систему не требуется. Для тестов выделена статическая библиотека **sniffer_lib** (все исходники проекта кроме `main.cpp`), с которой линкуются все тестовые бинарники.

```
# tests/CMakeLists.txt (упрощённо)
include(FetchContent)
FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG         v1.14.0
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

macro(add_gtest target source)
    add_executable(${target} ${source})
    target_link_libraries(${target} PRIVATE sniffer_lib GTest::gtest_main)
    include(GoogleTest)
    gtest_discover_tests(${target})
endmacro()

add_gtest(test_PacketParser      test_PacketParser.cpp)
add_gtest(test_PacketCapture    test_PacketCapture.cpp)
# ... и ещё 4 тестовых цели
```

Листинг 1 — Подключение GoogleTest в tests/CMakeLists.txt

3. СТРУКТУРА ТЕСТОВ В ПРОЕКТЕ

```
tests/
├─ CMakeLists.txt           ← правила сборки (FetchContent + gtest)
├─ test_PacketParser.cpp    ← 9 тестов
├─ test_PacketCapture.cpp   ← 10 тестов
├─ test_PcapCaptureStrategy.cpp ← 9 тестов (контракт + USE_PCAP)
├─ test_TrafficStatistics.cpp ← 8 тестов
├─ test_ReportGenerator.cpp ← 7 тестов
├─ test_ConsoleUI.cpp       ← 7 тестов
└─ scenarios/
    ├─ scenario_mock_capture.cpp
    ├─ scenario_real_capture.cpp
    ├─ scenario_export_report.cpp
    └─ scenario_invalid_interface.cpp
```

Рисунок 1 — Структура директории tests/

4. МОДУЛЬНЫЕ ТЕСТЫ

Для каждого класса написан отдельный тестовый файл. Каждый тест-класс охватывает: корректный ввод, некорректный ввод, граничные случаи, пустые данные, неподдерживаемые протоколы и некорректные пакеты.

Класс / Файл	Тестов	Что проверяется (примеры)
PacketParser	9	mock TCP/UDP/ICMP/ICMPv6; неизвестный протокол; реальные байты IPv4+IPv6; обр
PacketCapture (MockCaptureStrategy)	10	listInterfaces(); selectInterface(-1), (0), (99); startCapture без выбора; drain 3/30 пакетов; s
PcapCaptureStrategy (контракт ICaptureStrategy)	9	fresh.isRunning() == false; listInterfaces не пустой; select valid/neg; start без select; stop; фа
TrafficStatistics	8	начальные нули; инкремент пакетов и байт; агрегация по IP; отдельный подсчёт TCP/
ReportGenerator	7	getOutputPath(); путь по умолчанию; файл создаётся; содержит заголовок; содержит TC
ConsoleUI	7	конструктор; "0" → выход; опция 1 → "INTERFACES"; нечисловой ввод; опция 5 без ,

Таблица 1 — Сводная таблица модульных тестов

Пример: тест PacketParser (реальные байты)

```
// Вспомогательная функция строит Ethernet II + IPv4-кадр в памяти
RawPacket makeRealIPv4(uint8_t proto, const uint8_t src[4], const uint8_t dst[4]) {
    std::vector<uint8_t> frame;
    frame.insert(frame.end(), 6, 0x00);           // dst MAC
    frame.insert(frame.end(), 6, 0x00);           // src MAC
    frame.push_back(0x08); frame.push_back(0x00); // EtherType = IPv4
    frame.push_back(0x45);                         // Version=4, IHL=5 (20 байт)
    frame.insert(frame.end(), 8, 0x00);           // padding полей до proto
    frame.push_back(proto);                        // [9] — IANA протокол
    frame.push_back(0x00); frame.push_back(0x00); // checksum
    frame.insert(frame.end(), src, src + 4);       // [12..15] src IP
    frame.insert(frame.end(), dst, dst + 4);       // [16..19] dst IP
    RawPacket p; p.isMock = false; p.data = frame; p.length = frame.size();
    return p;
}

TEST_F(PacketParserTest, RealIPv4TcpFrame) {
    const uint8_t src[4] = {192, 168, 1, 10}, dst[4] = {8, 8, 8, 8};
    PacketInfo info = parser.parse(makeRealIPv4(6, src, dst));
    EXPECT_EQ(info.protocol, Protocol::TCP);
    EXPECT_EQ(info.sourceIp, "192.168.1.10");
    EXPECT_EQ(info.destinationIp, "8.8.8.8");
}
```

Листинг 2 — Тест разбора реального IPv4-кадра

Пример: тест TrafficStatistics (агрегация по IP)

```

TEST_F(TrafficStatisticsTest, SameSourceIpAggregated) {
    stats.addPacket(makePacket("10.0.0.1", "10.0.0.2", Protocol::TCP, 100));
    stats.addPacket(makePacket("10.0.0.1", "10.0.0.3", Protocol::UDP, 200));

    const auto& ip = stats.getIpStats();
    ASSERT_EQ(ip.count("10.0.0.1"), 1u);           // одна запись
    EXPECT_EQ(ip.at("10.0.0.1").packetCount, 2u); // два пакета
    EXPECT_EQ(ip.at("10.0.0.1").totalBytes, 300u); // 100 + 200 байт
}

```

Листинг 3 — Тест агрегации TrafficStatistics по IP-адресу

Пример: тест ConsoleUI (перенаправление потоков)

```

// RAII-обёртки для перенаправления std::cin / std::cout
struct StdinRedirect {
    explicit StdinRedirect(const std::string& text)
        : buf_(text), old_(std::cin.rdbuf(buf_.rdbuf())) {}
    ~StdinRedirect() { std::cin.rdbuf(old_); }
    std::istreambuf& buf_;
};

struct StdoutCapture {
    StdoutCapture() : old_(std::cout.rdbuf(buf_.rdbuf())) {}
    ~StdoutCapture() { std::cout.rdbuf(old_); }
    std::string str() const { return buf_.str(); }
    std::ostreambuf& buf_;
};

TEST_F(ConsoleUITest, RunExitsOnZero) {
    ConsoleUI ui(*capture_, parser_, stats_, *reporter_);
    StdinRedirect in("0"); // имитируем ввод "0" (выход из меню)
    StdoutCapture out;
    EXPECT_NO_THROW(ui.run());
    EXPECT_NE(out.str().find("Exiting"), std::string::npos);
}

```

Листинг 4 — Тест ConsoleUI с перенаправлением stdin/stdout

5. СЦЕНАРНЫЕ ПРОГРАММЫ

В дополнение к модульным тестам разработаны четыре сценарных исполняемых файла (с собственным *main()*), зарегистрированных в CTest через *add_test()*. При обнаружении ошибки программа выводит *[FAIL]* и завершается с кодом 1.

Сценарий	Что проверяется
scenario_mock_capture	Список и выбор интерфейса; запуск/остановка захвата; разбор 30 пакетов TCP/UDP
scenario_real_capture	Реальный захват через PcapCaptureStrategy при наличии libpcap. При нехватке памяти
scenario_export_report	Полный захват → запись отчёта в /tmp/ → проверка содержимого файла (заголовки)
scenario_invalid_interface	Негативный и выходящий за диапазон индекс; захват без выбора интерфейса; getI

Таблица 2 — Сценарные программы

```
// Фрагмент scenario_invalid_interface.cpp
// Тест: передача nullptr в конструктор PacketCapture → std::invalid_argument
bool threw = false;
try { PacketCapture cap(nullptr); }
catch (const std::invalid_argument&) { threw = true; }
if (!threw) { std::cerr << "[FAIL] Should throw.\n"; return 1; }
std::cout << "[OK] Null strategy throws std::invalid_argument.\n";
```

Листинг 5 — Фрагмент сценарной программы обработки ошибок

6. ПРИМЕРЫ ВЫПОЛНЕНИЯ ТЕСТОВ

Запуск отдельного бинарника

```
$ ./build/tests/test_TrafficStatistics
[=====] Running 8 tests from 1 test suite.
[ RUN      ] TrafficStatisticsTest.InitiallyEmpty
[ OK       ] TrafficStatisticsTest.InitiallyEmpty (0 ms)
[ RUN      ] TrafficStatisticsTest.AddPacketIncrementsTotalCount
[ OK       ] TrafficStatisticsTest.AddPacketIncrementsTotalCount (0 ms)
[ RUN      ] TrafficStatisticsTest.SameSourceIpAggregated
[ OK       ] TrafficStatisticsTest.SameSourceIpAggregated (0 ms)
[ RUN      ] TrafficStatisticsTest.ProtocolCountsSeparated
[ OK       ] TrafficStatisticsTest.ProtocolCountsSeparated (0 ms)
[ RUN      ] TrafficStatisticsTest.ResetClearsEverything
[ OK       ] TrafficStatisticsTest.ResetClearsEverything (0 ms)
...
[ PASSED   ] 8 tests.
```

Листинг 6 — Вывод test_TrafficStatistics

Полный прогон через CTest

```
$ ctest --test-dir build --output-on-failure
Test project NetworkPacketSniffer/build
 1/57 PacketParserTest.MockTcpPacket ..... Passed 0.00 sec
 2/57 PacketParserTest.MockUdpPacket ..... Passed 0.00 sec
 ...
 9/57 PacketParserTest.ProtocolToStringAllValues ... Passed 0.00 sec
10/57 PacketCaptureTest.ListInterfacesNonEmpty ..... Passed 0.00 sec
 ...
19/57 MockCaptureStrategyTest.Delivers30Packets .... Passed 1.62 sec
 ...
32/57 TrafficStatisticsTest.InitiallyEmpty ..... Passed 0.00 sec
 ...
47/57 ConsoleUITest.ConstructorNoThrow ..... Passed 0.00 sec
 ...
54/57 scenario_mock_capture ..... Passed 1.61 sec
55/57 scenario_real_capture ..... Passed 0.66 sec
56/57 scenario_export_report ..... Passed 1.61 sec
57/57 scenario_invalid_interface ..... Passed 0.01 sec

100% tests passed, 0 tests failed out of 57
Total Test time (real) = 5.71 sec
```

Листинг 7 — Полный прогон: 57/57 тестов пройдено за 5.71 с

7. ЦЕЛИ ТЕСТОВОГО ПОКРЫТИЯ

Каждый метод тестируется по шести категориям ввода:

- **Корректный ввод** — ожидаемый результат при нормальных данных. *Пример: `proto=6` → `Protocol::TCP`*
- **Некорректный ввод** — невалидные значения. *Пример: `selectInterface(-1)` → `false`*
- **Граничные случаи** — минимальные/максимальные значения. *Пример: фрейм ровно 14 байт (Ethernet header)*
- **Пустые данные** — нулевые структуры, пустые коллекции. *Пример: `TrafficStatistics` без `addPacket()` → 0 пакетов*
- **Неподдерживаемые протоколы** — IANA-номера вне таблицы. *Пример: `proto=253` → `Protocol::UNKNOWN`*
- **Некорректные пакеты** — обрезанные буферы. *Пример: `isMock=false`, `data` пустой → `Protocol::UNKNOWN`*

8. КОМАНДЫ СБОРКИ И ЗАПУСКА ТЕСТОВ

```

# Шаг 1: конфигурация (GoogleTest скачается автоматически при первом запуске)
cmake -S . -B build

# Шаг 2: сборка всего проекта — приложение + все тесты + сценарии
cmake --build build

# Шаг 3: запуск всех тестов
ctest --test-dir build

# Подробный вывод / вывод только при падении теста
ctest --test-dir build --verbose
ctest --test-dir build --output-on-failure

# Запуск только конкретного набора тестов
ctest --test-dir build -R PacketParser      # тесты PacketParser
ctest --test-dir build -R TrafficStatistics # тесты TrafficStatistics
ctest --test-dir build -R scenario          # только сценарные программы

# Прямой запуск бинарника GoogleTest (с цветным форматированием)
./build/tests/test_PacketParser
./build/tests/test_ConsoleUI

# Прямой запуск сценарных программ
./build/tests/scenario_export_report
./build/tests/scenario_invalid_interface

```

Листинг 8 — Все команды для сборки и запуска тестов

Требования к окружению

Инструмент	Версия	Примечание
CMake	≥ 3.14	FetchContent_MakeAvailable
C++ компилятор	$\text{GCC} \geq 8 / \text{Clang} \geq 7$	C++17, std::filesystem
Интернет	1-я сборка	GoogleTest v1.14.0 скачается автоматически
libsar	Опционально	Без него: mock, тесты rsar пропускаются

Таблица 3 — Требования к окружению

9. ЗАКЛЮЧЕНИЕ

К проекту **NetworkPacketSniffer** добавлена полноценная система тестирования на базе GoogleTest v1.14.0, полностью интегрированная в CMake через механизм FetchContent.

Результаты тестирования

Метрика	Значение
Всего тестов (CTest)	57
Пройдено	57 (100%)
Провалено	0
Общее время	5.71 с
Файлов: unit-тесты	6
Файлов: сценарии	4
Покрытых классов	6

Таблица 4 — Итоги прогона тестов

Выполненные работы

- Выделена **sniffer_lib** — статическая библиотека для линковки тестов.
- **53 модульных теста** в 6 файлах для всех ключевых классов.
- **4 сценарных программы** проверяют полный сквозной путь исполнения.
- Тестирование ConsoleUI реализовано без изменения исходного кода — за счёт перенаправления `std::cin / std::cout`.
- Все 57 тестов проходят за **5.7 секунды** командой `ctest --test-dir build`.

Все требования практического задания №4 выполнены в полном объёме.